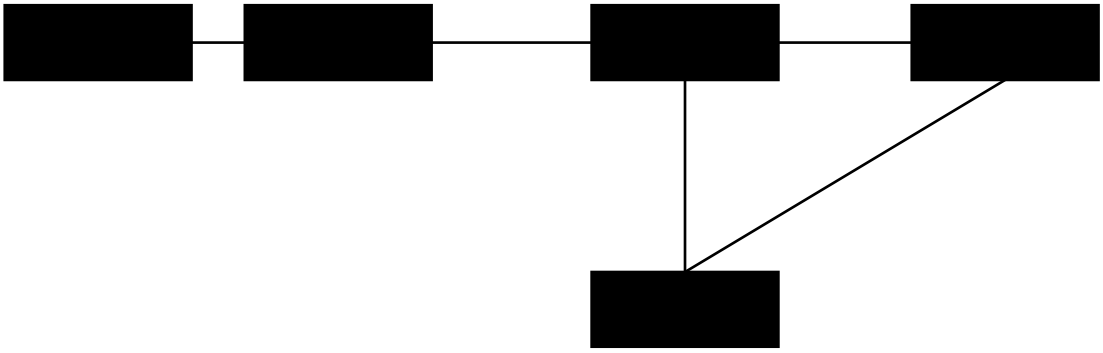
Distributed Key-Value Store (Amazon Dynamo Style)
Comprehensive HLD Study Guide

This guide consolidates the design goals, architecture, core distributed systems concepts, and interview questions for designing a distributed Key-Value database similar to Amazon Dynamo.



1. Introduction

Key-value stores provide constant-time style access by storing values against unique keys. They are ideal for carts, sessions, preferences and caching.

Example: Key=UserID, Value={ProductID→Quantity}.

2. Goals

Scalability through partitioning.

High availability through replication.

Decentralization using coordinator + gossip.

Eventual consistency using vector clocks.

Low latency using partitioning and quorums.

3. Partitioning

Use consistent hashing instead of hash(key)%N because adding a server moves only nearby keys.

Virtual nodes distribute load evenly across physical machines.

4. Replication

Replication factor N stores copies on multiple servers.

Coordinator computes preference list and forwards requests.

Replicas improve availability, durability and read scalability.

5. GET/PUT

PUT waits for W acknowledgements.

GET reads R replicas and resolves conflicts if necessary.

Typical values: N=3,R=2,W=2.

6. Quorum

R+W>N ensures read/write overlap, improving freshness while allowing configurable latency.

7. Vector Clocks

Track causal history rather than timestamps.

Detect concurrent writes and enable application-level conflict resolution.

8. Gossip

Nodes periodically exchange membership metadata, enabling decentralized failure detection and partition ownership discovery.

9. Merkle Trees

Tree of hashes used for anti-entropy repair. Only mismatched branches are synchronized, saving bandwidth.

10. Failure Handling

Replica failure: continue if quorum satisfied.

Coordinator failure: any node can coordinate.

Network partition: AP trade-off under CAP theorem.

Interview Questions & Answers

Question	Answer
Why KV DB?	Simple key lookup, no joins, horizontal scalability.
Why UserID partition?	Entire cart stays together.
Why consistent hashing?	Minimal data movement on scaling.
What are virtual nodes?	Multiple logical positions per server for load balancing.
Why replication?	Availability and durability.
Explain R+W>N	Overlapping quorums improve consistency.
Why eventual consistency?	Better availability and latency.
Why not timestamps?	Clock skew; vector clocks track causality.
What is gossip?	Decentralized membership dissemination.
What is Merkle Tree?	Efficient replica synchronization.

Coordinator crash?	Another node takes over.
Replica down?	Proceed if quorum met; repair later.
CAP choice?	Availability + Partition Tolerance.

Interview Answer Flow: Requirements → Data Model → Partitioning → Replication → GET/PUT → Quorum → Vector Clocks → Gossip → Merkle Trees → Trade-offs.